

Analysis: Collecting Cards

This problem requires some basic knowledge of probability and combinatorics. We want to calculate the expected number of packs we need to buy to obtain all C different cards.

Let's denote by $E(x)$ the expected number of packs we would need to buy if we started with x different cards (it doesn't matter what those cards are). The answer to the problem is the value for $E(0)$. We also know that $E(C) = 0$, because if we already have C different cards we don't need to buy any additional packs.

We can derive useful equations for other values of $E(x)$ by thinking about all the possible outcomes after buying one additional pack. Let's call $T(x,y)$ the probability of ending up with y different cards after opening a new pack. Then we have the following equation for $E(x)$:

$$E(x) = 1 + \sum_{y=x}^{\min(C, x+N)} T(x, y) \cdot E(y)$$

We need to buy at least one new pack, so that's where the 1 comes from. The expected number of packs we need to buy after that depends on how many new cards we get. If we end up with y different cards we need to add the expected number of packs to reach C starting from y , which we called $E(y)$, multiplied by the probability of this particular alternative given by $T(x, y)$.

All these equations put together form a system of linear equations with an upper triangular matrix which can be solved using the standard [back substitution](#) method.

But we still haven't said how to calculate the entries of the matrix T (that is, the values of $T(x, y)$ for all different x and y). We'll calculate this with the help of [binomial coefficients](#): the number of different possible packs is $\binom{C}{N}$. To end up with y different cards, we need to choose $y-x$ out of $C-x$ possible new cards, and the remaining $N-(y-x)$ have to be chosen from the x cards we already have. The answer then is:

$$T(x, y) = \frac{\binom{C-x}{y-x} \cdot \binom{x}{N-(y-x)}}{\binom{C}{N}}$$

(For those with some knowledge of probability, this is called the [hypergeometric distribution](#)).

The special case, where $N = 1$, is the well known [coupon collector's problem](#).

Analysis: Crossing the Road

A grid of roads; a person moving from place to place; this problem has all the hallmarks of a graph problem, and a shortest path problem at that. Our pedestrian is trying to get from one corner of the map to the opposite corner. Some steps that she takes can be made at any time, and take 2 minutes; for others, she has to wait until the light turns green, and then take another minute. How can we minimize the time she takes to reach her goal?

To solve this problem, we need to describe the state of the world. The pedestrian can be in any of $2m$ positions in the x direction, in any of $2n$ positions in the y direction, and at some time t . Since the pedestrian can always just wait if she needs to, she would always prefer to be at (x, y, t) over $(x, y, t+1)$; so an algorithm for computing the *earliest* time at which the pedestrian can arrive at (x, y) will solve this problem.

This problem differs from a standard graph problem in that, for two neighboring locations across a road from one another, the weight of the edge between them is not fixed. As it turns out this is only a minor complication, and we can use slightly modified versions of some standard shortest-path algorithms.

From each location, the pedestrian can go north, south, east or west, unless the direction in question is off the edge of the map. The amount of time this takes will be 2 minutes if it's along a block, or (the amount of time until the traffic light is green) + 1 minute if it's to cross a road.

Bellman-Ford is probably the easiest algorithm to implement for this. The longest path will take at most $O(m+n)$ steps, so your algorithm will terminate in at most that many stages. At each stage, for each point, you'll try to update its (up to) four neighbors. The running time is $O(nm(n+m))$.

Dijkstra's algorithm is usually more efficient. With $4nm$ states and 4 directions in which the pedestrian can move from each state, this algorithm will complete in $O(nm \log(nm))$ time.

The correctness of either algorithm can be proved in almost the same way as its standard counterpart. We leave these proofs as worthy exercises for the reader.

More information:

[Dijkstra's algorithm](#) - [Bellman-Ford algorithm](#)

Analysis: Multi-base happiness

How can you tell if a number is unhappy? You can tell if it enters a cycle when you apply the process described in the problem. Here is one example, in base 10:

42 → 20 → 4 → 16 → 37 → 58 → 89 → 145 → 42 repeated

Naturally, $x \rightarrow y$ denotes that y is the next number if we apply the process on x . Starting from any number, if you apply the process, eventually you will reach 1 or enter one of those cycles.

But is that really true? A careful reader might ask: Couldn't the process keep hitting new numbers, and never enter a cycle? Look at the cycle for 42: Before you hit 42 again, the numbers just jump around without a clear pattern.

It turns out that this can never happen: that there are only finitely many such cycles, and they're all finite in length. In fact, in base 10, this is the only one! All the numbers involved in such a cycle must be reasonably small. Indeed, when you start a number that is big (think about 99999..9999), applying the process will lead to numbers that become smaller and smaller very rapidly. One can easily prove that, in any base B , there is a threshold H of $O(B^3)$ such that any number larger than H will have a smaller successor.

Another question is: Given a set of bases, does there exist a number that is happy in all of them? We don't know the answer to that question, but based on our computation we know such numbers exist for all bases up to 10. On the other hand, if you intuit that the property of a number being happy is *somehow* random, and *somehow* independent across different bases, then you can believe that multi-base happiness is rare, and that the density of such numbers decreases exponentially with the number of bases. In our problem. The smallest happy number for the input (2, 3, 4, 5, 6, 7, 8, 9, 10) is 11814485; just barely affordable with a brute-force search.

In the computation, one obvious trick is to cache, for a pair (x, B) , whether x is happy in base B , so we can avoid following the jumping sequence or the cycles every time. We only need to do this for small values of x -- for example all $x \leq 1000$ would be more than enough -- since any 32-bit integer bigger than 10000 becomes smaller than 1000 in one step.

Since the maximum base is 10, one may realize that there are only $2^9 - 10 = 502$ possible distinct inputs. So why don't we just calculate them all? This can actually be faster than solving the input cases one by one, if we solve the smaller sets first. For a set S of bases, we do not need to start the search from 2; we can start from the answer for any $S' \subset S$, since a number happy in all bases from S is at least happy in all bases from S' .

If you think your implementation is still not fast enough. Then run your program while you are solving other problems. There are only 502 possible input cases. Solve all of them, produce the list of answers, and then start the submission process; just don't forget you also need to submit the slow program that produced the list. This is why we had a special note at the bottom of the problem statement.

More Information

Wikipedia article: [Happy Numbers](#)